

SQL Important Concepts

DBMS 40-Day Challenge • High-Yield Revision Notes

EXAM READY

1 IN and NOT IN Operators

The **IN** operator allows you to specify multiple values in a **WHERE** clause. It acts as a shorthand for multiple **OR** conditions.

♦ IN Operator

Selects rows where the column value matches **any** value in the list.

```
SELECT * FROM Emp
WHERE Address IN ('Delhi', 'Chandigarh',
                 'Pune');
```

Meaning: Select employees residing in Delhi *OR* Chandigarh *OR* Pune.

♦ NOT IN Operator

Excludes rows where the column value matches **any** value in the list.

```
SELECT * FROM Emp
WHERE Address NOT IN ('Delhi', 'Chandigarh',
                     'Pune');
```

Meaning: Select employees NOT residing in Delhi, Chandigarh, or Pune.

Example Table: Emp

Eid	Name	Address	Output for IN ('Delhi', 'Chandigarh', 'Pune')
1	Ravi	Chandigarh	Selected (Matches Chandigarh)
2	Varun	Delhi	Selected (Matches Delhi)
3	Nitin	Pune	Selected (Matches Pune)
4	Robin	Bangalore	Excluded (Selected in NOT IN)

★ **Remember:** **IN** = Included in list | **NOT IN** = Excluded from list.

2 IN with Subqueries

IN is heavily used with nested subqueries where the inner query returns a list of values to the outer query.

Question: Find employees who are currently assigned to at least one project.

```
SELECT Ename FROM Emp
WHERE Eid IN (
    SELECT Eid FROM Project
);
```

Execution Steps (Inner → Outer):

- Inner Query Executes:** `SELECT Eid FROM Project;` (e.g., returns result set: `{1, 3, 4, 5}`).
- Outer Query Evaluates:** `SELECT Ename FROM Emp WHERE Eid IN (1, 3, 4, 5);`
- Result:** Employees whose `Eid` is present in the subquery result set are selected.

3 EXISTS and NOT EXISTS Operators

EXISTS and **NOT EXISTS** test for the existence of rows in a subquery. They return a Boolean value (**TRUE** or **FALSE**) and are primarily used with **correlated subqueries**.

4 LIKE Operator (Pattern Matching)

The `LIKE` operator is used in a `WHERE` clause to search for a specified pattern in a column using two wildcards:

Symbol	Meaning	Example
<code>%</code>	Represents zero, one, or multiple characters	<code>'A%'</code> matches "A", "Amrit", "Arun"
<code>_</code>	Represents exactly one single character	<code>'_A%'</code> matches "Varun", "Raj"

Common Wildcard Patterns & Examples

1. Starts with 'A': `LIKE 'A%'`

```
SELECT * FROM Emp WHERE Name LIKE 'A%';
```

Matches: Arun, Amrit, Ajeet

2. Ends with 'N': `LIKE '%N'`

```
SELECT * FROM Emp WHERE Name LIKE '%N';
```

Matches: Varun, Arun

3. Contains 'ee': `LIKE '%ee%'`

```
SELECT * FROM Emp WHERE Name LIKE '%ee%';
```

Matches: Sangeeta, Geeta

4. Second character is 'A': `LIKE '_A%'`

```
SELECT * FROM Emp WHERE Name LIKE '_A%';
```

Matches: Varun (Varun), Ramesh (Ramesh)

5. Exactly 5 chars & 2nd character is 'A': `LIKE '_A____'`

```
SELECT * FROM Emp WHERE Name LIKE '_A____';
```

Pattern: `_ A _ _ _` (Total length = 5)

★ LIKE Pattern Cheat Sheet

- `'A%'` : Starts with A
- `'%A'` : Ends with A
- `'%A%'` : Contains A anywhere
- `'_A%'` : 'A' is in the 2nd position
- `'__A%'` : 'A' is in the 3rd position
- `'_A____'` : Exactly 5 characters, 'A' is 2nd

5 SQL SEQUENCE

A `SEQUENCE` is a database object used to automatically generate unique numeric values (commonly used for primary keys like Employee ID, Order ID, Transaction ID).

Creation Syntax

```
CREATE SEQUENCE student_seq
START WITH 1
INCREMENT BY 1;
```

Fetching Next Value:

```
SELECT student_seq.NEXTVAL FROM dual;
-- Returns: 1, then 2, then 3...
```

Sequence Properties

Property	Meaning
<code>START WITH</code>	First value to be generated
<code>INCREMENT BY</code>	Step difference between values
<code>MAXVALUE</code>	Maximum limit allowed
<code>CYCLE / NOCYCLE</code>	Restart or stop after reaching limit

Example Sequence:

```
CREATE SEQUENCE seq1 START WITH 100 INCREMENT BY 5 MAXVALUE 200 NOCYCLE;
```

Generated Output sequence: 100, 105, 110, 115, ..., 200 .

6 SQL Query Execution Order ★★★★★

Although SQL queries are written in a specific syntactical sequence, the database engine evaluates clauses in a distinct **logical execution order**.

Syntactic Order (Written)

1. SELECT
2. FROM
3. WHERE
4. GROUP BY
5. HAVING
6. ORDER BY

Logical Execution Order (Executed)

1. FROM & JOINS
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. DISTINCT & ORDER BY / LIMIT

Logical Execution Pipeline

STEP 1	STEP 2	STEP 3	STEP 4	STEP 5	STEP 6	STEP 7	STEP 8
FROM / JOIN	WHERE	GROUP BY	HAVING	SELECT	DISTINCT	ORDER BY	LIMIT



Easy Memory Trick:

F - J - W - G - H - S - D - O - L

(From → Join → Where → Group BY → Having → Select → Distinct → Order BY → Limit)

Execution Walkthrough Example

```
SELECT Department, COUNT(*)
FROM Employee
WHERE City = 'Delhi'
GROUP BY Department
HAVING COUNT(*) < 5
ORDER BY COUNT(*) DESC
LIMIT 1;
```

Execution Order	Clause Evaluated	Operation Performed
1. FROM	FROM Employee	Fetches raw data from the Employee table.
2. WHERE	WHERE City = 'Delhi'	Filters rows; retains only employees located in Delhi.
3. GROUP BY	GROUP BY Department	Groups remaining rows by Department values.
4. HAVING	HAVING COUNT(*) < 5	Filters grouped data; keeps departments with < 5 staff.
5. SELECT	SELECT Department...	Computes expressions & chooses output columns.
6. ORDER BY	ORDER BY COUNT(*) DESC	Sorts output groups in descending order of staff count.
7. LIMIT	LIMIT 1	Returns only the top 1 row from the sorted result set.

7 Aggregate Functions & NULL Behavior ★★

SQL aggregate functions perform a calculation on a set of values and return a single scalar value. The 5 major aggregate functions are `MAX()`, `MIN()`, `COUNT()`, `SUM()`, and `AVG()`.

🚨 CRITICAL CONCEPT: What is NULL?

- `NULL` represents an *unknown, missing, or unavailable* value.
- `NULL ≠ 0` and `NULL ≠ ""` (Empty string).
- Golden Rule:** `COUNT(*)` counts all rows (including NULLs). **All other aggregate functions completely IGNORE NULL values.**

Sample Dataset for Analysis (Table: T1)

Row ID	Column A	Column B	Function Call	Result	Explanation
1	1	NULL	<code>COUNT(*)</code>	5	Counts total rows including NULLs
2	2	NULL	<code>COUNT(A)</code>	4	Ignores 1 NULL in Column A
3	3	NULL	<code>COUNT(B)</code>	0	All values are NULL in Column B
4	4	NULL	<code>SUM(A)</code>	10	1 + 2 + 3 + 4 = 10 (Ignores NULL)
5	NULL	NULL	<code>AVG(A)</code>	2.5	SUM(10) / COUNT(4) = 2.5
			<code>SUM(B) / AVG(B)</code>	NULL	All-NULL column yields NULL

DISTINCT with Aggregate Functions

The `DISTINCT` keyword forces aggregate functions to consider only **unique non-NULL values**.

Given Salaries: {10000, 20000, 30000, 30000, 50000, NULL}

- `COUNT(Salary)` → 5 (Counts all non-null values).
- `COUNT(DISTINCT Salary)` → 4 (Unique set: {10000, 20000, 30000, 50000}).
- `AVG(DISTINCT Salary)` → (10000 + 20000 + 30000 + 50000) / 4 = 27,500.

8 Department-wise Highest Salary Query

A classic exam/interview problem: Find employees earning the highest salary in each department.

Step 1: Get Max Salary Per Dept

```
SELECT Department, MAX(Salary)
FROM Employee
GROUP BY Department;
```

Step 2: Get Employee Details

```
SELECT Ename, Department, Salary
FROM Employee
WHERE Salary IN (
    SELECT MAX(Salary)
    FROM Employee
    GROUP BY Department
);
```

🚨 **Common Mistake:** Use `WHERE Salary IN (...)` instead of `WHERE Salary = (...)` because the subquery returns **multiple rows** (one max salary per department).